

阅游 V1.1.0 赛博朋克智能听读系统技术设计说明书

软件基本信息

软件名称：阅游

版本号：V1.1.0

开发者：胡传龙

开发语言：Dart 3.x (Flutter 3.x 客户端) / Go 1.21 (服务端)

运行平台：Android 5.0 (API 21) 及以上

软件定位：赛博朋克视觉风格的沉浸式本地小说听读者，集成 2048 益智游戏，支持双轨并行云端 TTS、本地 TTS 降级、算法合成粉噪声背景音和物理 Chirp 音效，全部核心业务在 Flutter 端实现，服务端仅承担 TTS 音频地址中转和默认书籍资源分发。

整体架构设计

阅游采用 Feature-Driven Clean Architecture，代码按功能模块纵向切分，每个模块内部严格遵循 domain → providers → presentation 的横向分层。core/ 层提供全局主题、配置和工具，不引入任何业务功能代码；features/*/domain/ 层只允许纯 Dart 逻辑，严禁引入 Flutter UI 库；features/*/providers/ 层使用 Riverpod 管理异步状态；features/*/presentation/ 层只消费状态，禁止编写业务逻辑。

客户端层次关系

presentation (UI渲染, 只消费状态)

↓ 读取

providers (Riverpod状态机, 业务编排)

↓ 调用

domain / services (纯Dart业务逻辑, 无UI依赖)

↓ 使用

core (主题/配置/日志/存储, 全局基础设施)

Go 服务端

main.go → Gin Router

handler_tts.go TTS 音频 URL 中转

handler_book.go 默认书籍目录和章节地址

handler_privacy.go 隐私政策 HTML 页面

↓

OSS/CDN 资源层 (音频文件、章节文本)

本地与远端的边界非常清晰：用户的书架数据、阅读进度、游戏存档、TTS 设置均只存入本设备的 SharedPreferences，服务端只负责音频 URL 的一次性中转，不保存任何与用户身份有关的信息。

TTS 双轨并行听读系统

分离下载契约

阅游 TTS 严格遵循"分离下载"协议，禁止将 HTTP POST 响应体直接作为音频数据使用。完整流程如下：

第一步：ReaderProvider 取出当前待朗读句子文本

第二步：TtsEngineService 向 Go 服务端发起 POST /tts

请求体：{"text": "句子文本", "voice": "音色名称"}

第三步：服务端调用阿里云 TTS API，将音频文件存入 OSS

返回 JSON：{"status": "success", "url": "https://oss.example.com/xxx.mp3"}

第四步：客户端解析 url 字段，使用独立 HTTP GET 请求下载 MP3 文件

第五步：写入本地缓存目录（app_support/tts_cache/），再交给 audioplayers 播放

这种设计的核心价值在于：客户端永远不保存 POST 响应体的二进制数据，只保存从 CDN 直接下载的音频文件，完全规避了因网络 MTU 分包导致的音频损坏问题，同时也使缓存文件可以独立校验完整性。

双轨并行架构（Producer-Consumer）

TtsAudioNotifier（位于 lib/features/audio/providers/tts_audio_notifier.dart）实现了双轨并行的生产者-消费者模型：

- 预加载轨道（_prefetchRunner）：后台协程持续从 TtsSentenceSource（由 ReaderProvider 注册）取出待朗读句子，向 TtsEngineService 发起下载，将结果推入 TtsAudioBuffer（六槽有界队列）。队列满时主动等待，形成背压控制，避免过度预取消耗流量。
- 播放轨道（_playRunner）：串行从 TtsAudioBuffer 取出已下载的音频项，通过 TtsEngineService 播放，播完即从缓冲区移除（"播完即焚"）。
- 两条轨道通过共享的 TtsAudioBuffer 实例通信，互不阻塞，互不感知对方的具体实现。

预加载轨道

```
ReaderProvider.getSentence()  
    → TtsEngineService.downloadAudio()  
    → TtsAudioBuffer.push()    ← 满则等待（背压）
```

播放轨道

```
TtsAudioBuffer.take()    ← 空则等待  
    → TtsEngineService.play()  
    → 播放完成 → 取下一条
```

会话哨兵机制

TtsAudioNotifier 内部维护一个整型 _session 计数器。每次用户切换音色、跳转章节、停止朗读时，_session 递增一次。预加载轨道和播放轨道在每次异步操作前都会比对本地保存的 sessionId 与当前 _session：如果不一致，则立即退出，不再推进任何后续步骤。

这一设计彻底解决了音色切换后旧音色的音频"幽灵播放"问题：即使旧轨道的下载任务已经完成并写入本地文件，只要 _session 已递增，旧轨道的所有后续操作都会被安静地废弃，不会影响新轨道的启动。

本地 TTS 降级

当云端 TTS 连续失败次数超过阈值（后台宽容模式下阈值提高），TtsFallbackController 自动切换到系统 FlutterTts（本地 TTS 引擎）进行降级朗读。降级后用户仍可收听，不至于因网络问题导致完全无法使用听读功能。降级状态下页面会展示提示信息，网络恢复后可手动重置回云端模式。

状态机流转

TTS 的生命周期由 TtsAudioState（密封类，位于 lib/features/audio/domain/tts_audio_state.dart）建模：

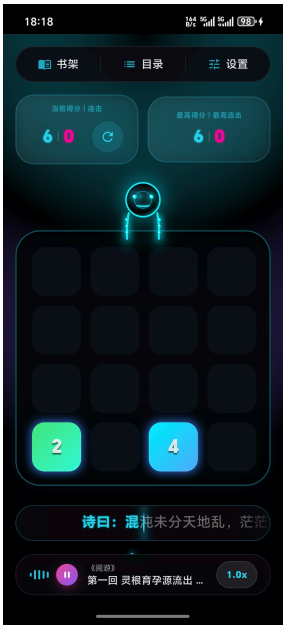


图7 TTS播放状态

图7展示听读状态时的播放控制区域，包含进度指示、播放/暂停按钮和当前朗读句子高亮。

2048 游戏引擎

全盘求和得分法

阅游 2048 刻意偏离了主流 2048 实现的“累加得分法”（每次合并时把合并后数值累加到得分），改用“全盘求和法”：每次有效滑动完成后，updateScore() 遍历棋盘全部 16 格，将所有现存方块的数值相加作为当前分数。这与原版 Web 端完全一致，保证了从 Web 移植到 Flutter 时玩家的分数感知不变。

updateScore() 实现逻辑：

```
total = 0
for row in [0..3]:
  for col in [0..3]:
    if board[row][col] != null:
      total += board[row][col].value
score = total
if score > bestScore: bestScore = score
```

移动与合并算法

移动方向由 Direction 枚举表示（up / down / left / right）。执行 move(direction) 时：

1. 根据方向计算移动向量（如 up 对应 {x:0, y:-1}）。
2. 对行列进行方向感知的遍历排序（向下时行序反转，向右时列序反转）。
3. 对每个方块沿向量方向滑动，找到最远可达的空格或可合并目标。
4. 合并条件：目标格有方块、值相等、且目标格本轮尚未被合并（mergedIds 集合记录已合并 ID 防止连锁合并）。
5. 合并后触发 SfxService.playMoveFeedback() 按合并值选择对应 Chirp 音效阶段。
6. 遍历完成后如有任何格移动，则随机生成新方块（2 的概率 90%，4 的概率 10%），调用 updateScore() 刷新分数，检查 _movesAvailable() 判断是否游戏结束。

游戏状态（棋盘、分数、最佳分数、Combo 计数）在 App 切入后台（didChangeAppLifecycleState: paused）时通过防抖 Timer 持久化到 StorageService，下次启动自动恢复。

黑客后门彩蛋

GameProvider 实现了 eliminateTileById(int id) 方法，供 BoardMascot 在触发彩蛋动画后调用。彩蛋触发条件：连续点击吉祥物 8 次，触发三阶段动画（膨胀→坍缩→粉红霓虹粒子消散），然后调用 eliminateTileById 删除指定方块，删除后立即调用 updateScore() 同步刷新全盘分数，避免 UI 显示虚高的旧分数。

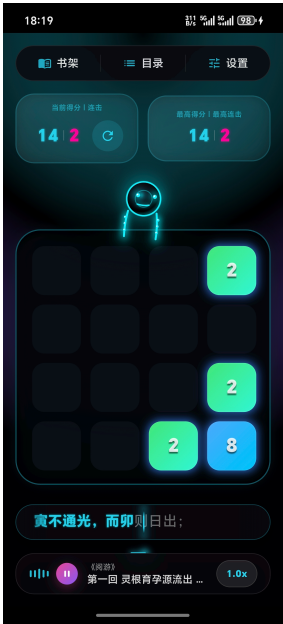


图9 2048游戏界面

图9展示2048游戏棋盘、吉祥物和得分区域。

Chirp 物理音效系统

解析式扫频公式

SfxService（位于 lib/features/audio/services/sfx_service.dart）在应用启动时预合成 4 个 WAV 音效片段，全部基于解析式 Chirp 相位公式，保证零相位突变、零杂音：

$$\phi(t) = 2\pi \times (f \times t + (f - f) \times t / (2T))$$

$$\text{样本 } y(t) = \sin(\phi(t)) \times \text{音量} \times \text{指数衰减}(e^{-(\text{decayRate} \times t)})$$

其中 f 为起始频率，f 为终止频率，T 为扫频时长，t 为当前时间（秒）。

四阶段递进参数

根据合并后方块数值分为 4 个阶段：

阶段	触发数值	f (Hz)	f (Hz)	扫频时长	音量	总时长
一	≤ 16	440	880	100ms	0.12	300ms
二	≤ 128	400	900	110ms	0.15	340ms
三	≤ 1024	350	950	120ms	0.18	380ms
四	> 1024	330	1000	130ms	0.22	420ms

WAV 文件在 SfxService.init() 时全部预合成存入 _tierWavs 列表，播放时直接从内存取用，无磁盘 I/O 延迟，确保合并音效与视觉动画同帧触发。

算法合成环境背景音

Voss-McCartney 粉噪声合成

AmbientService（位于 lib/features/audio/services/ambient_service.dart）采用 Voss-McCartney 算法生成粉噪声（Pink Noise，即 $1/f$ 噪声）：

粉噪声通过对 N 级白噪声序列按各自更新频率叠加产生， N 级中第 k 级每 2^k 个样本更新一次，所有级别的均值随机数求和后归一化，得到在对数频谱上接近平坦（即各倍频程能量相等）的宽带噪声。与白噪声相比，粉噪声对人耳更为舒适，常用于掩蔽环境噪音和提升专注力。

两种氛围风格

阅游提供两种环境音风格，均完全通过算法实时合成，不依赖任何外部音频资源文件：

- wuxia（武侠/自然）风格：粉噪声基底叠加低幅随机瞬态，模拟山林风声和远处虫鸣的质感。
- warm（温馨/城市）风格：粉噪声基底叠加 60Hz 正弦工频嗡鸣（模拟日光灯管电子噪声），营造室内暖灯氛围。

生成的 WAV 数据通过 audioplayers 的 BytesSource 接口以 ReleaseMode.loop 循环播放，音频焦点设置为 AndroidAudioFocus.none，不与 TTS 竞争系统音频焦点，两者可同时发声。

文本解析引擎

Isolate 后台处理

TextParser（位于 lib/features/reader/domain/text_parser.dart）使用 Isolate.run() 在独立后台线程执行解析，防止大文件（>100KB）的 UTF-8 解码和正则匹配阻塞主线程导致 UI 卡顿。

解析规则

解析器对原始 TXT 文本执行以下处理：

1. UTF-8 解码，去除 BOM，规范化换行符（\r\n 和 \r 统一转为 \n）。
2. 识别章节标题：匹配“第X章”/“第X节”/“Chapter X”等模式，标题行单独作为一个句子，用于章节目录构建。
3. 段落拆分：按双换行符切分段落，过滤纯空行和纯符号行（如 --- 分隔线）。
4. 句子切分：按中文句末标点（。！？…）和英文句末标点（. !?）切分，保留标点附属于前一个句子。
5. 长句紧急拆分：单句超过 120 字符时，进行二次拆分（按逗号或停顿符号），防止单句过长导致 TTS 朗读截断。
6. 空句过滤：结果中剔除内容为空或仅含空白的句子项。

解析结果以 List<ReaderSentenceCursor> 格式返回，每个 cursor 记录章节索引、句子索引和句子文本，供 ReaderProvider 和 TtsAudioNotifier 按索引消费。

阅读器与阅读进度管理

阅读状态管理

ReaderProvider（位于 lib/features/reader/providers/reader_provider.dart）继承 ChangeNotifier，维护以下核心状态：

- 当前书籍信息（BookModel）
- 章节列表（从服务端或本地解析获取）
- 当前章节句子列表（List<String>）
- 当前高亮句子索引（currentSentenceIndex）
- 章节加载状态（枚举 ChapterLoadState）

进度持久化

阅读进度通过 StorageService 写入 SharedPreferences，以书籍 ID 为 key 存储 {chapterIndex, sentenceIndex}。应用重启后，ReaderProvider 在 _loadChapter() 时自动读取存档恢复。同一本书的进度独立存储，互不干扰。

与 TTS 联动

ReaderProvider 实现了 TtsSentenceSource 接口，在 TtsAudioNotifier 通过 registerSentenceSource() 注册后，由预加载轨道持续调用 getSentence() 取出待朗读句子。TTS 每播完一句，TtsAudioNotifier 回调 onSentenceComplete()，ReaderProvider 将 currentSentenceIndex 加一并同步高亮下一句，同时异步将新进度写入本地存储。

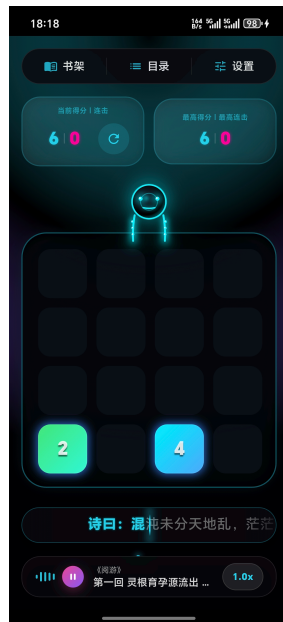


图6 阅读页面

图6展示阅读页面，当前高亮句子与 TTS 播放同步，已读句子和待读句子通过颜色区分。

书库管理

书库模块（lib/features/library/）管理用户导入的本地 TXT 文件和默认书籍入口。

用户导入 TXT 文件时, FileImportService 通过 Android file_picker 插件调用系统文件选择器, 仅获取用户主动授权的单个文件, 不扫描其他目录, 不请求存储读取权限。文件内容读取后由 TextParser 在后台 Isolate 解析, 解析完成后将书名(取自文件名去扩展名)和本地文件路径写入 BookshelfProvider, 通过 StorageService 持久化到本地。

默认书籍(《西游记》)由 DefaultBookService 管理, 初次访问时向服务端请求章节目录和章节内容 URL, 下载并缓存到本地, 此后章节翻阅直接读缓存, 无需再次联网。

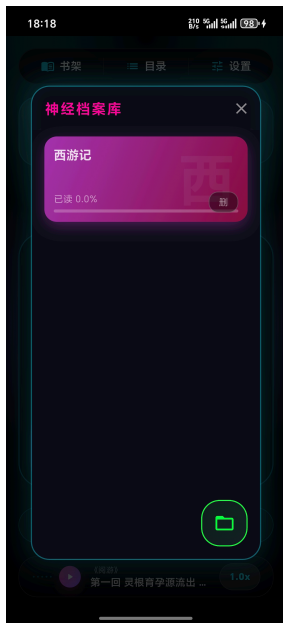


图4 书库页面

图4展示书库页面, 包含本地导入书籍和默认书籍入口。

主界面与吉祥物

主界面(DashboardScreen)是软件的核心枢纽, 集成了 TTS 播放控制区、章节快捷入口、设置入口和 2048 游戏棋盘。整个界面使用赛博朋克配色(以霓虹青 #00FFFF、霓虹绿 #39FF14、暗色背景 #0A0A0F 为主)。

BoardMascot(吉祥物组件, 位于 lib/features/game_2048/presentation/widgets/board_mascot.dart)趴在 2048 棋盘上方, 通过六层独立动画控制器实现丰富的表情交互:

- 眼球轨道: AnimationController 驱动 Offset 动画, 跟随玩家最后一次滑动方向(up/down/left/right 各对应不同偏移量), 滑动后 600ms 插值回中心, 可被新方向随时打断。
- 头部倾斜: 复用眼球轨道同步触发, 左右滑动分别倾斜 $\pm 6^\circ$, 增强身体感知动效。
- 身体跳跃: 合并高数值方块时触发 AnimationController 驱动的弹跳曲线, 可重启不排队。
- 表情层: -1.0 = 难过(游戏结束), 0.0 = 正常, 1.0 = 开心(合并大数值), 通过 Animation<double> 驱动嘴角弯曲程度的自定义 CustomPainter 绘制。
- 棋盘晃动(wobble): 每次有效滑动触发微幅 Offset 震动, 模拟吉祥物被棋盘晃动的物理效果。
- 独立眨眼: Timer 随机触发间隔(2~5 秒), 与其他动画层完全解耦, 全程循环。

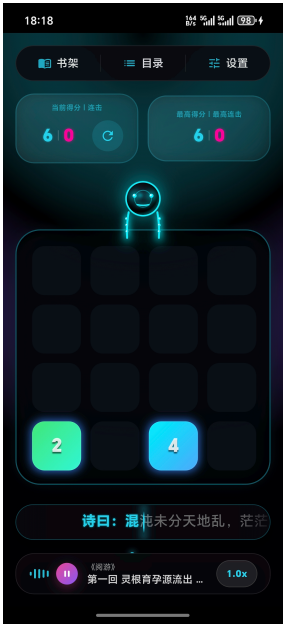


图3 主界面与2048区域

图3展示主界面布局，吉祥物、棋盘、TTS 控制区和设置入口在界面中的位置。

启动流程与隐私保护

启动初始化序列

```
main() 启动入口
  ↓ WidgetsFlutterBinding.ensureInitialized()
  ↓ StorageService.init()           本地 SharedPreferences 初始化
  ↓ SfxService.init()               预合成 4 阶段 Chirp WAV
  ↓ AmbientService.init()           初始化环境音播放器
  ↓ 检查 privacyAgreed 字段
    否 → 展示隐私弹窗，用户同意后继续
    是 → 直接进入主界面 (DashboardScreen)
  ↓ runApp(ProviderScope(child: YueyouApp()))
```

隐私弹窗一旦被拒绝，系统调用 `SystemNavigator.pop()` 退出，不进入任何功能界面。用户同意后写入 `privacyAgreed = true` 到 `SharedPreferences`，此后重启不再弹窗。设置页面提供"隐私政策"入口，用户可随时重新查看。



图1 启动页界面

图1展示软件启动页视觉入口。



图2 首次启动隐私政策弹窗

图2展示首次启动时的隐私政策弹窗界面。

隐私保护策略

软件不申请以下权限：通讯录、短信、通话记录、摄像头、麦克风、精确位置。全部用户数据（书架、阅读进度、游戏存档、TTS 设置）只存储在本设备的 SharedPreferences 和应用私有目录（app_support/tts_cache/），不向服务端同步，服务端日志中不记录任何与用户身份关联的信息。

Go 服务端接口

服务端位于 server/ 目录，使用 Gin 框架，共提供以下 4 类接口：

POST /tts: 接收 {"text":"...", "voice":"..."} 请求体, 调用阿里云 TTS API 生成音频并上传至 OSS, 返回 {"status":"success", "url":"..."} 响应 JSON。关键安全设计: AK 密钥通过 systemd EnvironmentFile 注入, 不写入代码仓库。

GET /book/:id/toc: 返回指定书籍 (当前为《西游记》ID=1) 的章节目录 JSON 数组, 每项包含章节编号和标题。

GET /book/:id/chapter/:cid: 返回指定章节的 CDN 可访问地址, 客户端再 GET 下载章节 TXT 文本内容。

GET /privacy: 返回隐私政策 HTML 页面, 供用户查看, 也作为应用商店审核的可访问链接提供。

设置系统

设置项通过 SettingsProvider (Riverpod ChangeNotifierProvider) 管理, 所有设置项读写都经由 StorageService 持久化, UI 层通过 ref.watch(settingsProvider) 响应变化。主要设置项:

- autoPlay: 章节加载完毕后是否自动开始 TTS 朗读 (默认关)
- selectedVoice: 云端 TTS 音色选择, 改变时触发 TtsAudioNotifier._session 递增, 废除旧音色的已缓冲音频
- ambientEnabled / ambientStyle / ambientVolume: 环境音开关、风格 (wuxia/warm) 和音量 (0.0~1.0)
- silencePauseSeconds: TTS 播放时检测到长时间静音 (无新句子) 后自动暂停等待的秒数
- sfxEnabled: 系统音效开关 (合并音效 Chirp + 触觉反馈)

所有设置项的展示文案统一收敛在 lib/features/settings/constants/settings_texts.dart, 不在 UI 文件中散落硬编码字符串。



图10 设置页面

图10展示设置页面, 包含朗读参数、环境音、音效和隐私政策入口的集中管理界面。



图11 设置页隐私政策入口

图11展示设置页中的隐私政策入口，用户可随时重新查看隐私政策全文。

测试体系

项目自动化测试位于 test/ 目录，覆盖以下核心场景：

文本解析测试：验证解析结果不丢失有效字符、原始顺序保持一致、连续标点不产生空句子，以及超长句子能被紧急拆分到合理长度范围内。

TTS 契约测试：验证 TtsEngineService 的 HTTP 请求符合"先 POST 获取 URL、再 GET 下载 MP3"的分离下载契约，模拟网络失败场景验证降级路径被正确触发。

音频状态机测试：验证 TtsAudioNotifier 的状态流转（Idle→Buffering→Playing→Paused→Playing），以及会话哨兵机制在切换音色时能正确废止旧会话的所有操作。

游戏引擎测试：验证全盘求和得分法的正确性（与累加得分法的差异），验证各方向移动后棋盘状态与预期一致，验证游戏结束判断（_movesAvailable() 返回 false 时 isOver 为 true）。

客户端静态分析要求 flutter analyze 零错误零警告，运行时控制台零 warning 输出。服务端通过 go test ./... 覆盖基础接口逻辑。

章节导航

章节列表页面（ChapterListScreen）在阅读时可从主界面入口访问，展示当前书籍全部章节的编号和标题，支持直接跳转到目标章节，跳转后 ReaderProvider 重新加载目标章节内容并重置 currentSentenceIndex 为 0。

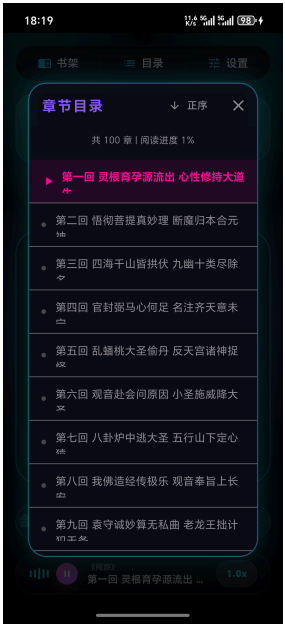


图8 章节列表页面

图8展示章节列表导航界面。